

## Laborator nr. 8

# Operații pe câmpuri de tip vector în MongoDB

Titular disciplină  
șef lucr. dr. ing. Cristian Aflori  
[cristian.aflori@academic.tuiasi.ro](mailto:cristian.aflori@academic.tuiasi.ro)

Titular aplicații  
șef lucr. dr. ing. Cătălin Mironeanu  
[catalin.mironeanu@academic.tuiasi.ro](mailto:catalin.mironeanu@academic.tuiasi.ro)

Titular aplicații, ing. Elena-Denisa Chelarasu  
[elena-denisa.chelarasu@academic.tuiasi.ro](mailto:elena-denisa.chelarasu@academic.tuiasi.ro)

Titular aplicații, ing. Ioana-Cristina Prioteasa  
[ioana-cristina.prioteasa@academic.tuiasi.ro](mailto:ioana-cristina.prioteasa@academic.tuiasi.ro)

an universitar 2025 - 2026



Obiective:

- Operatori pentru vectori: `$unwind`, `$push`, `$addToSet`
- Operatori condiționali: `$cond`, `$switch`
- Operatori dinamici: `$map`, `$reduce`, `$filter`
- Actualizarea vectorilor

## 1 Interogarea datelor folosind operatori pentru vectori

Vectorii în MongoDB sunt structuri de date puternice, iar în acest laborator se introduc operatorii de interogare pe vectori în *framework*-ul de agregare [1]. Acești operatori permit manipularea și remodelarea dinamică a vectorilor în documente, pregătind terenul pentru analize de date mai complexe.

Operatorul `$unwind` ia un câmp de tip vector și îl despachetează, creând pentru fiecare element un document separat. Imaginați-vă un document al unui student cu materiile într-o matrice. Aplicarea operatorului `$unwind` îl transformă în documente separate pentru fiecare materie.

Se consideră colecția `students` ca în exemplul de mai jos:

```
db.getCollection('students').insert(
{
  "_id": 1,
  "name": "Ionescu",
  "score": 85,
  "subjects": [
    "Math",
    "Physics",
    "Chemistry"
  ]
})
```

Aplicarea operatorului `$unwind` (referința bibliografică [2]):

```
db.getCollection("students").aggregate([
  { $unwind: "$subjects" }
])
```

În cadrul *framework*-ului de agregare se pot folosi operatorii condiționali pentru a selecta date în funcție de criterii dinamice.

Operatorul `$cond` [3] permite implementarea expresiilor condiționale. De exemplu, se poate evalua dacă un student a fost admis sau nu în funcție de scorul obținut, ca în exemplul următor:

```
db.getCollection("students").aggregate([
{
  $project: {
    name : 1,
    score : 1,
    result : {
      $cond : {
        if : { $gte : ["score" , 50] },
        then : "Pass",
        else : "Fail"
      }
    }
  }
}]
```

Pentru implementare unei operații de tip switch-case este operatorul `$switch`. Acesta permite evaluarea condițiilor multiple și alegerea rezultatului corespunzător [4].

```
db.getCollection("students").aggregate([
{
  $project: {
    "name": 1,
    "grade": {
      $switch: {
        branches: [
          { case: { $gte: ["$score", 90] }, then: "A" },
          { case: { $gte: ["$score", 80] }, then: "B" },
          { case: { $gte: ["$score", 70] }, then: "C" },
          { case: { $gte: ["$score", 60] }, then: "D" },
          { case: true, then: "F" }
        ]
      }
    }
  }
}]
```

Pentru filtrarea și gestionarea câmpurilor de tip vector se folosesc operatori dinamici: `$map`, `$reduce` și `$filter`.

Operatorul `$map` permite transformări asupra câmpurilor de tip vector [5]. El permite specificarea unei logici de efectuat pentru fiecare element din vector pe care operatorul îl iterează, returnând un vector nou ca rezultat final. Operatorul `$map` expune conținutul elementului curent al vectorului printr-o variabilă specială, cu numele implicit `$$this`.

De exemplu, adăugați la colecția `students` un atribut nou - `marks` de tip vector, ce conține valori între 1 și 10, de exemplu `"marks": [5, 4, 7]`.

Folosind operatorul `$map` se afișează un atribut nou - `newMarks`, ce conține valorile din `marks` mărite cu 1.

```
db.getCollection("students").aggregate([
{
  $project: {
    name: 1,
    marks: 1,
    newMarks: {
      $map: {
        input: "$marks",
        as: "rawMarks",
        in: { $add : ['$$rawMarks', 1] }
      }
    }
  }
}]
```

Cu ajutorul operatorului `$reduce`, puteți specifica o logică de executat pentru fiecare element dintr-un câmp de tip vector pe care operatorul îl iterează, dar în loc să returnați un vector ca rezultat final, returnați o singură valoare [6]. De obicei, se folosește `$reduce` pentru a calcula un rezultat după ce se analizează fiecare element al vectorului. De exemplu, puteți dori să returnați un număr înmulțind împreună o valoare de câmp specifică din fiecare element din vector. La fel ca și operatorul `$map`, operatorul `$reduce` furnizează logicii problemei acces la elementul curent al matricei prin intermediul variabilei `$$this`. Operatorul furnizează, de asemenea, o a doua variabilă, numită `$$value`, pentru ca logica problemei să o actualizeze atunci când acumulați rezultatul unic (de exemplu, rezultatul înmulțirii).

De exemplu, se consideră colecția `orders` ce conține comenzi de forma:

```
{ "orderId": "ORD1",  
  "products": [  
    "Laptop",  
    "Kettle",  
    "Phone",  
    "Microwave"  
  ]  
}
```

Vrem să afișăm un câmp nou `productList` care să conțină lista produselor din `products`, produsele fiind delimitate de ";":

```
db.getCollection("orders").aggregate([  
  { "$set": {  
    "productList": {  
      "$reduce": {  
        "input": "$products",  
        "initialValue": "",  
        "in": {  
          "$concat": ["$$value", "$$this", "; "]  
        }  
      }  
    }  
  } }  
])
```

Operatorul `$filter` selectează dinamic elemente dintr-un câmp de tip vector pe baza unor condiții specificate. Este util atunci când trebuie să recuperăm elemente specifice în funcție de anumite criterii [7].

De exemplu, afișarea unui câmp nou `newMarks` care conține doar notele mai mari ca 5:

```
db.getCollection("students").aggregate([  
  {  
    $project: {  
      name : 1,  
      marks : 1,  
      newMarks: {  
        $filter : {  
          input : "$marks",  
          as : "rawMarks",  
          cond : { $gte : ['$rawMarks', 5] }  
        }  
      }  
    }  
  }  
])
```

## 2 Actualizarea datelor stocate în câmpuri de tip vectori

Pentru actualizarea documentelor într-o colecție, de obicei se folosesc instrucțiunile `updateOne` și `updateMany`. Pentru actualizarea câmpurilor de tip vector se folosesc următorii operatori:

| Nume             | Descriere                                                                                                                                                                                             |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| \$               | Funcționează ca un substituent pentru a actualiza primul element care se potrivește condiției interogării.                                                                                            |
| \$[]             | Funcționează ca un substituent pentru a actualiza toate elementele dintr-un vector pentru documentele care se potrivesc condiției interogării.                                                        |
| \$[<identifier>] | Funcționează ca un substituent pentru a actualiza toate elementele vectorului care se potrivesc condiției din <code>arrayFilters</code> , pentru documentele care se potrivesc condiției interogării. |
| \$push           | Adaugă un element în câmpul de tip vector.                                                                                                                                                            |
| \$addToSet       | Adaugă elemente la câmp de tip vector numai dacă acestea nu există deja în set.                                                                                                                       |
| \$pop            | Șterge primul sau ultimul element din vector.                                                                                                                                                         |
| \$pull           | Șterge toate elementele din vector care se potrivesc condiției interogării.                                                                                                                           |
| \$pullAll        | Șterge toate elementele din vector care se potrivesc cu o listă de valori.                                                                                                                            |

Tabela 1. Operatori pentru actualizarea câmpurilor de tip vector

### 2.1 Operatorii \$push, \$addToSet, \$pop, \$pull, \$pullAll

Operatorul `$push` este folosit pentru adăugarea elementelor la un vector. Acesta este folosit în principal în operațiile de actualizare și de inserare în loc de agregare [8].

Să presupunem că avem un vector de materii în documentul unui student și dorim să adăugăm o nouă materie acelui student. În acest caz, vom folosi `$push` pentru a adăuga nouă materie în vectorul de materii.

```
db.getCollection("students").updateOne(  
  { _id: 1 },  
  { $push: { subjects: "English" } }  
)
```

Pentru a adăuga elemente unice într-un vector se folosește operatorul `$addToSet` [9]:

```
db.getCollection("students").updateOne(  
  { _id: 1 },  
  { $addToSet: { subjects: "English" } }  
)
```

Operatorul `$pop` elimină primul sau ultimul element a unei matrice. Operatorul `$pop` cu valoarea de -1 elimină primul element al vectorului și cu 1 elimină ultimul element din vector.

```
db.getCollection("students").updateOne(  
  { _id: 1 },  
  { $pop: { marks: -1 } }  
)
```

Operatorul `$pull` elimină din vector toate instanțele unei valori sau valorile care se potrivesc unei condiții specificate.

```
db.profiles.insertOne( { _id: 1, votes: [ 3, 5, 6, 7, 7, 8 ] } )  
  
db.profiles.updateOne( { _id: 1 }, { $pull: { votes: { $gte: 6 } } } )
```

Operatorul `$pullAll` elimină toate instanțele valorilor specificate din vector. Spre deosebire de operatorul `$pull`, care elimină elemente prin specificarea unei interogări, `$pullAll` elimină elementele care se potrivesc valorilor listate.

```
db.survey.insertOne( { _id: 1, scores: [ 0, 2, 5, 5, 1, 0 ] } )  
  
db.survey.updateOne( { _id: 1 }, { $pullAll: { scores: [ 0, 5 ] } } )
```

## 2.2 Operatorul \$

Operatorul pozițional \$ identifică un element dintr-un vector pentru a fi actualizat fără a specifica explicit poziția elementului în vector.

Sintaxa:

```
db.collection.updateOne(  
  { <array>: value ... },  
  { <update operator>: { "<array>.$": value } }  
)
```

Se consideră colecția `students`:

```
db.students.insertMany([  
  { "_id": 1, "grades": [ 85, 80, 80 ] },  
  { "_id": 2, "grades": [ 88, 90, 92 ] },  
  { "_id": 3, "grades": [ 85, 100, 90 ] }  
)
```

Vrem să înlocuim prima apariție a valorii 80 din vector cu 82:

```
db.students.updateOne(  
  { _id: 1, grades: 80 },  
  { $set: { "grades.$": 82 } }  
)
```

Rezultatul:

```
{ "_id": 1, "grades": [ 85, 82, 80 ] }  
{ "_id": 2, "grades": [ 88, 90, 92 ] }  
{ "_id": 3, "grades": [ 85, 100, 90 ] }
```

Operatorul pozițional \$ facilitează actualizările vectorilor care conțin sub-documente. Utilizați operatorul pozițional \$ pentru a accesa câmpurile din documentele încorporate cu notația punctată pe operatorul \$.

Sintaxa:

```
db.collection.updateOne(  
  { <query selector> },  
  { <update operator>: { "array.$.field": value } }  
)
```

Se consideră câmpul `grades` astfel:

```
{  
  "_id": 4,  
  "grades": [  
    { "grade": 80, "mean": 75, "std": 8 },  
    { "grade": 85, "mean": 90, "std": 5 },  
    { "grade": 85, "mean": 85, "std": 8 }  
  ]  
}
```

Pentru a actualiza cu valoarea 6 câmpul `std` al primului element al vectorului care se potrivește condiției `grade = 85`:

```
db.students.updateOne(  
  { _id: 4, "grades.grade": 85 },  
  { $set: { "grades.$.std": 6 } }  
)
```

Rezultatul:

```
{  
  "_id": 4,  
  "grades": [  
    { "grade": 80, "mean": 75, "std": 8 },  
    { "grade": 85, "mean": 90, "std": 6 },  
    { "grade": 85, "mean": 85, "std": 8 }  
  ]  
}
```

Operatorul \$ poate actualiza primul element al vectorului care se potrivește cu mai multe criterii de interogare specificate cu operatorul \$elemMatch.

Se consideră documentul:

```
{
  "_id": 5,
  "grades": [
    { "grade": 80, "mean": 75, "std": 8 },
    { "grade": 85, "mean": 90, "std": 5 },
    { "grade": 90, "mean": 85, "std": 3 }
  ]
}
```

În exemplul de mai jos, operatorul \$ actualizează valoarea câmpului std în primul sub-document care are câmpul grade ≤ 90 și un câmp mean > 80:

```
db.students.updateOne(
  {
    _id: 5,
    grades: { $elemMatch: { grade: { $lte: 90 }, mean: { $gt: 80 } } }
  },
  { $set: { "grades.$.std": 6 } }
)
```

Rezultatul:

```
{
  "_id": 5,
  "grades": [
    { "grade": 80, "mean": 75, "std": 8 },
    { "grade": 85, "mean": 90, "std": 6 },
    { "grade": 90, "mean": 85, "std": 3 }
  ]
}
```

## 2.3 Operatorul \$[]

Operatorul pozițional \$[] indică faptul că operatorul de actualizare ar trebui să modifice toate elementele din câmpul vectorului specificat.

Sintaxa:

```
db.collection.updateOne(
  { <query conditions> },
  { <update operator>: { "<array>.$[]": value } }
)
```

Se consideră documentele din colecția students (modificate conform primului exemplu din subcapitolul anterior):

```
{ "_id": 1, "grades": [ 85, 82, 80 ] },
{ "_id": 2, "grades": [ 88, 90, 92 ] },
{ "_id": 3, "grades": [ 85, 100, 90 ] }
```

Pentru a incrementa cu 5 valorile din grades pentru toate documentele din colecție:

```
db.students.updateMany(
  { },
  { $inc: { "grades.$[]": 5 } },
)
```

Operatorul pozițional \$[] facilitează actualizările într-un vector care conține sub-documente. Pentru a accesa câmpurile din documentele încorporate, utilizați notația cu punct și operatorul \$[]:

Sintaxa:

```
db.collection.updateOne(
  { <query selector> },
  { <update operator>: { "array.$[].field": value } }
)
```

Fie colecția `stud` și vrem să incrementăm cu 5 valorile `grade`:

```
db.stud.insertMany( [
  {
    "_id": 1,
    "grades": [
      { "grade": 80, "mean": 75, "std": 8 },
      { "grade": 85, "mean": 90, "std": 6 },
      { "grade": 85, "mean": 85, "std": 8 }
    ]
  },
  {
    "_id": 2,
    "grades": [
      { "grade": 90, "mean": 75, "std": 8 },
      { "grade": 87, "mean": 90, "std": 5 },
      { "grade": 85, "mean": 85, "std": 6 }
    ]
  }
] )

db.stud.updateMany(
  { },
  { $inc: { "grades.$.grade": 5 } }
)
```

Fie colecția `results` și vrem să creștem cu 5 valorile din `grades`, cu excepția celor care au valoarea 100:

```
db.results.insertMany( [
  { "_id": 1, "grades": [ 85, 82, 80 ] },
  { "_id": 2, "grades": [ 88, 90, 92 ] },
  { "_id": 3, "grades": [ 85, 100, 90 ] }
] )

db.results.updateMany(
  { "grades": { $ne: 100 } },
  { $inc: { "grades.$[]": 5 } }
)
```

## 2.4 Operatorul `$[<identifier>]`

Operatorul pozițional filtrat `$[<identifier>]` identifică elementele vectorului care se potrivesc condițiilor din `arrayFilters` pentru o operație de actualizare:

Sintaxa:

```
db.collection.updateMany(
  { <query conditions> },
  { <update operator>: { "<array>.$[<identifier>]": value } },
  { arrayFilters: [ { <identifier>: <condition> } ] }
)
```

Se consideră colecția `results` și vrem să setăm valorile mai mari sau egale cu 85 la 90:

```
{ "_id": 1, "grades": [ 85, 82, 80 ] },
{ "_id": 2, "grades": [ 88, 90, 92 ] },
{ "_id": 3, "grades": [ 85, 100, 90 ] }

db.results.updateMany(
  { },
  { $set: { "grades.$[element]": 90 } },
  { arrayFilters: [ { "element": { $gte: 85 } } ] }
)
```

Operatorul \$[<identifier>] facilitează actualizările vectorului care conține documente încorporate. Pentru a accesa câmpurile din documentele încorporate, utilizați notația cu punct și operatorul \$[<identifier>]:

Sintaxa:

```
db.collection.updateMany(  
  { <query selector> },  
  { <update operator>: { "array.$[<identifier>].field": value } },  
  { arrayFilters: [ { <identifier>: <condition> } ] }  
)
```

Se consideră colecția students:

```
{  
  "_id": 1,  
  "grades": [  
    { "grade": 80, "mean": 75, "std": 6 },  
    { "grade": 85, "mean": 100, "std": 4 },  
    { "grade": 85, "mean": 100, "std": 6 }  
  ]  
},  
{  
  "_id": 2,  
  "grades": [  
    { "grade": 90, "mean": 100, "std": 6 },  
    { "grade": 87, "mean": 100, "std": 3 },  
    { "grade": 85, "mean": 100, "std": 4 }  
  ]  
}
```

Vrem să setăm valoarea pentru mean la 90 pentru elementele din vector care au grade  $\geq 85$  și std  $\geq 6$ :

```
db.students.updateMany(  
  { },  
  { $set: { "grades.$[elem].mean" : 90 } },  
  { arrayFilters: [ { "elem.grade": { $gte: 85 }, "elem.std": { $gte: 6 } } ] }  
)
```

Rezultatul:

```
{  
  "_id": 1,  
  "grades": [  
    { "grade": 80, "mean": 75, "std": 6 },  
    { "grade": 85, "mean": 100, "std": 4 },  
    { "grade": 85, "mean": 90, "std": 6 }  
  ]  
},  
{  
  "_id": 2,  
  "grades": [  
    { "grade": 90, "mean": 90, "std": 6 },  
    { "grade": 87, "mean": 100, "std": 3 },  
    { "grade": 85, "mean": 100, "std": 4 }  
  ]  
}
```

### 3 Activități practice

Testați exemplele din laborator.

Creați colecția `studenti` cu următoarea structură:

- `_id`
- `nume`
- `prenume`
- `grupa`
- `skills` - de tip vector ce conține abilitățile studentului (ex: ["OOP", "programare web", "SQL", "MongoDB", "Windows", "Linux"])
- `catalog` - de tip vector de subdocumente cu următoarea structură:
  - `cod_disciplina` (ex: "ECBD")
  - `nr_absente` (ex: 1)
  - `note` - de tip vector (ex: [5, 7, 8, 9])

Folosind colecția `studenti`:

1. Populați colecția cu 3 studenți fără `skills` și fără `note` (din `catalog`).
2. Folosind operatorii specifici, adaugați elemente în `skills` și `catalog`.
3. Afișați descrescător clasamentul abilităților studenților (`skills`).
4. Folosind operatorii specifici, actualizați notele la disciplina "ECBD".
5. Afișați un câmp media la subdocumentul din `catalog`, câmp calculat ca media aritmetică a notelor la acea disciplină.  
(*Hint*: folosiți operatorul dinamic `$map`)
6. Afișați studenții care au mai mult de 2 absențe la "ECBD".
7. Afișați clasamentul mediilor studenților la "ECBD", ordonați pe grupe (`grupa`, `nume`, `prenume`, `media`).
8. Afișați clasamentul mediilor pe grupe la "ECBD" (`grupa`, `media`).

### Bibliografie

- [1] "MongoDB Documentation - Aggregation Operations."  
<https://www.mongodb.com/docs/manual/aggregation/>.
- [2] "MongoDB Documentation - \$unwind (aggregation)."  
<https://www.mongodb.com/docs/manual/reference/operator/aggregation/unwind/>.
- [3] "MongoDB Documentation - \$cond (aggregation)."  
<https://www.mongodb.com/docs/manual/reference/operator/aggregation/cond/>.
- [4] "MongoDB Documentation - \$switch (aggregation)."  
<https://www.mongodb.com/docs/manual/reference/operator/aggregation/switch/>.
- [5] "MongoDB Documentation - \$map (aggregation)."  
<https://www.mongodb.com/docs/manual/reference/operator/aggregation/map/>.
- [6] "MongoDB Documentation - \$reduce (aggregation)."  
<https://www.mongodb.com/docs/manual/reference/operator/aggregation/reduce/>.
- [7] "MongoDB Documentation - \$filter (aggregation)."  
<https://www.mongodb.com/docs/manual/reference/operator/aggregation/filter/>.
- [8] "MongoDB Documentation - \$push (aggregation)."  
<https://www.mongodb.com/docs/manual/reference/operator/aggregation/push/>.
- [9] "MongoDB Documentation - \$addToSet (aggregation)."  
<https://www.mongodb.com/docs/manual/reference/operator/aggregation/addToSet/>.